Rapport de Projet

WeatherScope

Big Data and NoSQL

Réalisé par
DITER Justin & SANCHEZ Louise

Encadré par
Mme. AKID-ZEKRI Hajer

Master 2 Informatique Parcours Systèmes Logiciels et Calculs Distribués
Université Côte d'Azur - Année 2026-2027

Résumé

Ce rapport présente ....

Abstract

This report studies .....

Table des matières

1 Introduction **4**

2 Périmètre Géographique, Temporalité & Provenance des Données **4**
2.1 Périmètre d'étude 4
2.2 Traçabilité, Traitement des Drapeaux (Flags) et Valeurs Manquantes 5

3 Architecture Système & Implémentation des Services **6**
3.1 Modèle de données 6
3.2 Services 7

4 Prototype distribué **7**
4.1 Déploiement : processus, ports et mémoire 8
4.2 Placement des observations 8
4.3 Préparation et alimentation des bases distribuées 9
4.4 Accès aux données et routage des requêtes 10
4.5 Perte d'une instance 11
4.6 Limites du prototype distribué 11

5 Ce que disent les hivers **12**

6 Conclusion **13**

A Anomalies relevées dans les données **14**

B Référence des services **15**
B.1 F1 : explorer une station 15
B.2 Traçabilité 16
B.3 F2 : épisodes prolongés 16
B.4 F3 : mois anormaux 17

1 Introduction

WeatherScope est un prototype d'analyse de données météo historiques, commandé par un observatoire régional. Il permet de choisir une zone et une période, de repérer des épisodes inhabituels (chaleur, fortes pluies), de comparer des zones, et de remonter à chaque fois jusqu'aux observations sources. Il tourne d'abord sur un seul serveur MongoDB, puis sur un cluster shardé.

Pour le mettre à l'épreuve, nous lui avons posé une question qui nous tient à cœur. L'un de nous a grandi à Grosse Pointe Woods, près de Détroit, entre 2000 et 2011 : des hivers blancs, des jours d'école annulés pour cause de neige. Aujourd'hui, la neige ne tient plus. Est-ce un souvenir embelli ou un vrai changement ? Et si l'hiver change, change-t-il de la même façon ailleurs ? Nous comparons le Michigan au sud de l'Allemagne : même bande de latitude, deux climats différents, l'un dominé par l'air arctique et la neige d'effet de lac des Grands Lacs, l'autre par l'air atlantique, plus doux.

Notre réponse, développée en section 5 : *l'hiver ne disparaît pas sous forme de neige qui tombe, mais de neige qui reste, et il disparaît le plus vite là où il était déjà le plus proche de zéro.* Pour le montrer, nous suivons le nombre de jours de neige au sol, la quantité de neige tombée et la température des hivers, de 1961 à 2020. Les services de WeatherScope couvrent toute l'année ; l'hiver est le fil rouge de ce rapport.

2 Périmètre Géographique, Temporalité & Provenance des Données

2.1 Périmètre d'étude

Données. Toutes les observations viennent de GHCN-Daily, l'archive quotidienne de la NOAA [1]. Les métadonnées ont été récupérées le 8 novembre 2026 ; l'URL, la date et le SHA-256 de chaque fichier sont dans `data/raw/metadata/MANIFEST.json`. La sélection se refait avec `scripts/select_stations.py`.

Zones. Le Michigan regroupe les stations américaines avec le code d'État `MI`. Le sud de l'Allemagne regroupe les stations situées en Bavière ou en Bade-Wurtemberg, d'après les frontières de Natural Earth [5]. Les deux zones ont une surface proche (environ 146 000 et 106 000 km^2). Nous écartons les stations allemandes au-dessus de 800 m (18 stations, dans les Alpes et la Forêt-Noire), car la neige y dépend surtout de l'altitude ; le Michigan ne dépasse pas 523 m. Les latitudes ne sont pas identiques : la moitié des stations du Michigan sont au sud de 43,0°N, la moitié des allemandes au nord de 48,8°N (figure 1).

Période. Nous étudions 1991–2020, la normale climatique actuelle de l'OMM [2]. La référence pour les anomalies (F3) est 1961–1990 ; elle passe par le même chargement mais est comptée à part. Un hiver va de décembre à février : 30 hivers, de 1990–91 à 2019–20.

Stations. Nous gardons les stations qui ont des données TMAX, TMIN ou PRCP sur 1991–2020 (tableau 1). Les réseaux diffèrent : au Michigan, 675 stations viennent de CoCoRaHS,

un réseau de bénévoles [3], et la plupart n'existent que depuis 2005. En Allemagne, le réseau (ECA&D [4]) reste stable. Une moyenne sur toutes les stations montrerait donc surtout l'arrivée de nouvelles stations. Pour les tendances, nous n'utilisons que les 233 stations « à longue série » : celles qui ont au moins 80 % de mois valides sur 1991–2020 pour TMAX, TMIN ou PRCP, un mois étant valide s'il lui manque au plus 5 jours, dont au plus 3 consécutifs (règle 3/5 de l'OMM [2]). L'inventaire de la NOAA en annonçait 269, mais il ne donne que la première et la dernière année et masque les trous (annexe A). Une fois chargées, les données comptent 5,1 millions de couples station–jour sur 1991–2020 (3,4 millions sur la référence), pour 200 000 demandés.

La hauteur de neige (SNWD) est disponible dans les deux zones. Les chutes de neige (SNOW) n'existent qu'au Michigan.

	Michigan	Sud de l'Allemagne
Stations de la zone	1 560	400
Stations retenues	1 058	295
dont à longue série (mesurée)	94	139
dont débutées en 2005 ou après	763	18
Stations retenues par élément : avec données / sur toute la période		
TMAX	317 / 95	264 / 93
TMIN	319 / 98	264 / 94
PRCP	1 045 / 117	294 / 148
SNOW	1 009 / 109	–
SNWD	890 / 110	287 / 140

Tableau 1: Stations retenues sur 1991–2020 (inventaire NOAA du 8/11/2026).

2.2 Traçabilité, Traitement des Drapeaux (Flags) et Valeurs Manquantes

Traçabilité. Chaque observation garde sa valeur brute, ses trois drapeaux, son heure d'observation, le fichier source et le lot d'import qui l'a écrite. Son identifiant est `station:date:élément` : pour tout résultat, on peut lister les relevés qui l'ont produit. Un import ne crée jamais de doublon : une valeur identique est ignorée, une valeur modifiée remplace l'ancienne, qui est conservée dans un historique. Chaque lot enregistre ses fichiers et leur SHA-256.

Drapeaux. Rien n'est supprimé au chargement ; les règles s'appliquent dans les requêtes. Une valeur avec un drapeau qualité (0,18 % des observations) est exclue des analyses. Le drapeau `P` (« manquant, présumé nul », 1,5 million de valeurs au Michigan) est traité comme une donnée manquante, pas comme un zéro. Une trace (`T`) compte comme un jour de précipitation de quantité négligeable. Les contrôles de la NOAA suffisent : nous n'avons trouvé aucune valeur aberrante ni aucun TMAX inférieur à TMIN sans drapeau (annexe A).

Valeurs manquantes. Nous ne comblons aucun trou. Un mois qui ne respecte pas la règle 3/5 n'entre pas dans les anomalies mensuelles (F3), et un jour manquant ou signalé

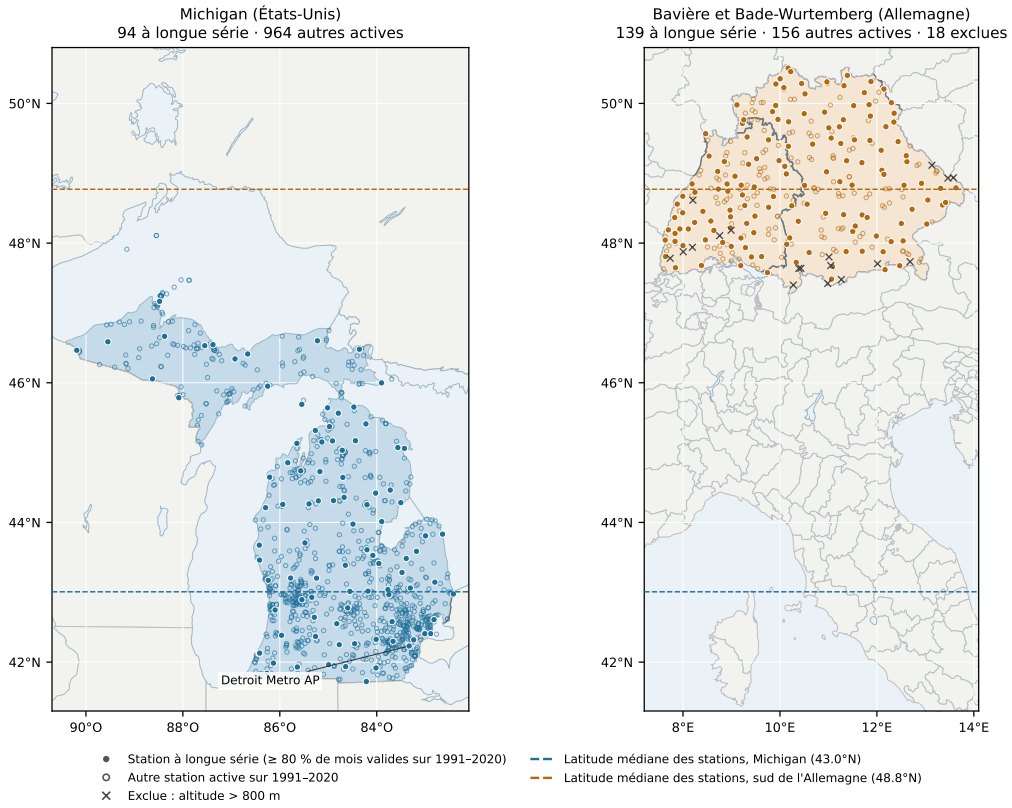


Figure 1: Stations retenues, à la même échelle. Pointillés : latitude médiane des stations de chaque zone.

interrompt un épisode (F2). Deux identifiants à moins d'un kilomètre l'un de l'autre (95 paires) comptent pour un seul site dans F3 et F4.

Heure d'observation. En Allemagne, ECA&D ne donne pas d'heure : chaque valeur couvre le jour climatologique du service météo national. Au Michigan, sur les 315 stations qui mesurent TMAX entre 1991 et 2020, 203 relèvent le matin (surtout à 7 h ou 8 h), 42 l'après-midi ou le soir, 28 à minuit (aéroports) et 42 ne donnent pas d'heure ; 118 ont changé d'heure au moins une fois. Pour une station relevée à 8 h, la TMAX notée le jour J est en général celle de l'après-midi de la veille. Deux stations voisines peuvent donc dater le même épisode avec un jour d'écart. F4 tolère ce décalage d'un jour entre stations, et les deux zones (UTC−5 et UTC+1) sont comparées sur des valeurs mensuelles ou saisonnières, jamais jour par jour.

3 Architecture Système & Implémentation des Services

3.1 Modèle de données

Les données sont dans MongoDB, avec un document par observation : une station, une date, un élément. L'identifiant `station:date:élément` sert à la fois de clé d'unicité pour les im-

ports (F5) et de référence pour la traçabilité. Quatre collections suffisent : `observations` (31,6 millions de documents), `stations` (avec un point GeoJSON), `history` (versions remplacées par un import) et `batches` (un document par import, avec ses fichiers et leurs SHA-256). Des validateurs `$jsonSchema` refusent tout document mal formé ; le détail des champs est dans `schema/README.md`. Trois index couvrent les accès : (station, élément, date) pour une station ; (zone, élément, date) pour les analyses régionales ; un index 2dsphere sur les stations pour les recherches géographiques.

3.2 Services

Les services sont regroupés dans un outil en ligne de commande, `python3 -m weatherscope`. Leurs entrées, sorties et exemples détaillés sont en annexe B ; nous ne reprenons ici que les choix qui comptent.

F1, explorer une station. Une agrégation par station et par période (mois, année, hiver) donne, à côté de chaque total, la part de jours renseignés et la validité du mois. Un cumul de pluie sur 17 % des jours n'a pas de sens : F1 le dit au lieu de le cacher.

Traçabilité. Tout résultat renvoie à ses relevés : la commande `trace` affiche, pour une station et une période ou une liste d'identifiants, la ligne NOAA d'origine avec ses drapeaux, son fichier et son lot d'import.

F2, épisodes prolongés. MongoDB ne renvoie que les jours qui satisfont la règle et dont la valeur est utilisable ; Python relie les jours consécutifs. Un jour absent ou signalé n'est jamais renvoyé, donc il coupe l'épisode au lieu de le prolonger, et chaque épisode dit pourquoi il s'arrête. Les épisodes suivent la suite des dates, pas l'année civile : la vague de froid du 30 décembre 2013 au 9 janvier 2014 à Détroit reste un seul épisode. La règle peut être un seuil fixe ou un percentile du climat de la station, parce que 30 °C n'a pas le même sens à Munich et à Marquette.

F3, mois anormaux. Chaque mois valide d'une station est comparé à sa normale 1961–1990, la normale précédente, de 30 ans comme le veut la convention [2] et indépendante de la période étudiée ; ses 13,5 millions d'observations sont comptées à part. Une station n'a de normale qu'avec 24 années valides sur 30. Pour que les grappes de stations autour de Détroit ne pèsent pas plus que tout le nord du Michigan, les anomalies sont moyennées par case de 1°, puis les cases entre elles selon leur surface ; deux stations à moins d'un kilomètre comptent pour une. F3 retrouve les mois attendus : mars 2012 au Michigan, août et juin 2003 en Allemagne. Une requête sur les deux zones et 60 ans prend environ 2 minutes : c'est la requête lourde de notre charge, et elle guide la suite de l'architecture.

4 Prototype distribué

La partie II conserve le modèle de données et le code des services de la partie I, mais répartit la seule collection volumineuse, `observations`, entre plusieurs serveurs MongoDB (*shards*)

réunis en un cluster. L'application ne voit qu'un seul point d'entrée et les requetes sont acheminées automatiquement vers les shards appropriés. Deux politiques de placement sont implémentées et comparées, les profils `hashed` et `zoned`.

4.1 Déploiement : processus, ports et mémoire

Le script `cluster.py` lance et surveille tous les processus (tableau 2). La commande `start --shards N` démarre le config server et N shards ; chacun est initialisé comme un replica set d'un seul membre (`replSetInitiate`), puis le script attend qu'il soit primaire. Il lance ensuite `mongos`, relié au config server, et enregistre chaque shard avec `addShard`. La commande ne relance que les processus qui ne répondent pas, ce qui nous permet d'avoir une restauration après une panne.

Tableau 2: Processus du déploiement distribué (N shards, $N \in \{2, 3\}$).

Processus	Port	Rôle	Dossier	Cache
`mongod` local	27017	serveur autonome de la partie I, référence	`cluster_data/local`	2 Go
config server	27200	métadonnées : shards, clés, chunks, zones	`cluster_data/config`	0,25 Go
`shard1RS`	27301	observations ; primary shard (métadonnées)	`cluster_data/shard1`	2/N Go
`shard2RS`	27302	observations	`cluster_data/shard2`	2/N Go
`shard3RS`	27303	observations (configuration à trois shards)	`cluster_data/shard3`	2/N Go
`mongos`	27400	routeur, seul point d'entrée des profils distribués	–	–

Les instances de données au sens du sujet sont les shards : ni `mongos` ni le config server ne stockent d'observation. Les replica sets d'un membre reprennent la simplification du lab 5 ; MongoDB les exige pour les shards et le config server, mais ils ne fournissent aucune copie des données.

Le cache WiredTiger total est fixé par `WS_CACHE_TOTAL_GB` (2 Go par défaut) et réparti entre les shards : 1 Go chacun avec deux shards, 0,67 Go avec trois. Le serveur local reçoit le même total, pour comparer les déploiements à budget de cache égal. Ce budget ne couvre pas tout : le config server et `mongos` consomment leur propre mémoire, et tous les processus partagent les mêmes cœurs, le même disque et le même cache de fichiers du système. Ce déploiement n'est donc pas équivalent à un cluster de machines indépendantes : il n'y a ni latence réseau ni pannes indépendantes.

4.2 Placement des observations

Seule la collection `observations` est partitionnée ; `stations`, `history` et `batches`, petites, restent entières sur le primary shard `shard1RS`. Chaque document est stocké entier sur un seul shard, et les chunks couvrent tout l'espace de la clé sans se chevaucher : la fragmentation

est horizontale, complète et disjointe. Comme `_id = station:date:élément` contient la station et que la clé de shard en dérive, une observation n'a qu'une destination possible. Il ne peut donc pas y avoir de doublon entre shards, alors que MongoDB ne garantit l'unicité de `_id` que shard par shard.

Profil `hashed`. La clé est `{station: "hashed"}`, avec 12 chunks initiaux répartis par le balancer. F1 reste ciblé sur un shard, et les écritures sont réparties quelle que soit la zone, alors qu'une clé sur la date concentrerait chaque import F5 sur le dernier chunk. En contrepartie, toute requête par zone ou par période est diffusée à tous les shards.

Profil `zoned`. La clé est `{area: 1, station: 1}` : la zone `michigan` est placée sur `shard1RS`, `southern_germany` sur `shard2RS`. Le champ `area`, recopié de la station au chargement, réalise une fragmentation dérivée. La station complète la clé, car `area` seule, avec deux valeurs, donnerait des chunks indivisibles. Les requêtes régionales sont ciblées et une panne reste confinée à sa zone. En revanche, l'équilibre dépend du volume de chaque zone (à vérifier avec `getShardDistribution`), et un troisième shard reste vide. Le tableau 3 résume les deux politiques.

Tableau 3: Comparaison des deux politiques de placement.

	`hashed`	`zoned`
Clé de shard	`{station: "hashed"}`	`{area: 1, station: 1}`
Placement décidé par	hachage, puis balancer	zones : allocation explicite
Requête sur une zone	diffusée à tous les shards	ciblée sur le shard de la zone
Écritures (F5)	réparties par station	dirigées vers le shard de chaque zone
Équilibre des données	indépendant des zones	dépend du volume de chaque zone
Un shard éteint	toute requête de zone est refusée	seule la zone touchée est refusée
Troisième shard	reçoit des chunks migrés	reste vide

4.3 Préparation et alimentation des bases distribuées

Avant tout chargement, `setup_db.py` prépare le profil : il supprime sa base sur le cluster si `--reset` est passé, active le sharding avec `shard1RS` comme primary shard, crée collections, validateurs et index, puis sharde `observations`. En profil `zoned`, il déclare aussi les zones.

Les données arrivent par deux voies. `sync_from_local.py` recopie les quatre collections du serveur local vers `mongos`. Il remplace chaque document ou l'insère s'il est absent : la copie est idempotente et le cluster contient exactement la référence locale. C'est un outil de migration, pas une mesure d'ingestion. Son option `--clear` vide les collections avec `delete_many`, ce qui conserve la configuration de sharding.

Pour F5, `load_observations.py` est lancé avec `--uri mongodb://localhost:27400`. Chaque document porte la clé de shard complète et n'est envoyé qu'à un shard ; le même

import est appliqué aux deux déploiements pour qu'ils restent identiques. L'historique (sur `shard1RS`) et l'observation (souvent sur un autre shard) sont écrits sans transaction distribuée. Un crash entre les deux se rattrape en relançant l'import, au prix d'une entrée d'historique en double.

4.4 Accès aux données et routage des requêtes

Toute l'application passe par la classe `Store` de `common.py`. Selon le profil, elle ouvre un seul client vers un seul point d'entrée : le serveur local (port 27017) ou `mongos` (port 27400). Elle ne se connecte jamais directement à un shard pour lire des données, ce qui assure la transparence de requête : les fonctions `f1` et `f3` de `services.py` reçoivent un `Store` et sont identiques pour les trois profils. Le client écrit avec `w="majority"` ; avec des replica sets d'un membre, cela revient à attendre la journalisation, mais le code est prêt pour de vrais réplicas. La commande `weather.py --profile local|hashed|zoned` expose F1 et F3 à l'utilisateur.

La méthode `scope()` construit tous les filtres sur `observations`. Elle exige une station ou une zone et refuse un filtre qui n'en contient pas, sauf si l'appelant passe `allow_broadcast=True`, comme F3 : une diffusion à tous les shards est ainsi toujours explicite. En profil `zoned`, elle ajoute la zone de chaque station demandée, lue dans un cache côté application, pour que le filtre fixe le préfixe de la clé. Le script `route.py` rend le routage observable : il construit un filtre avec `scope()`, demande son plan d'exécution à `mongos` et liste les shards contactés. Le tableau 4 donne le résultat pour trois requêtes représentatives.

Tableau 4: Shards contactés par des requêtes représentatives (deux shards).

Requête	Filtre construit par `scope()`	`hashed`	`zoned`
F1, une station, un an	`station`, `date` (et `area` en `zoned`)	1	1
F3, Michigan, 1991–1992	`area`, `element`, `qflag: null`, `date` de 1961 à 1992	2 (tous)	1
F3, deux zones, 1991–1992	`area` parmi les deux zones, mêmes autres critères	2 (tous)	2

Le filtre de F3 est le même dans les deux profils : seule la clé de shard change le routage. Pour F1, un seul shard répond et `mongos` relaie sa réponse. Pour F3, chaque shard contacté filtre ses documents et les trie par station puis date ; `mongos` fusionne ces flux triés en un seul (étape `SHARD_MERGE_SORT` du plan). L'application calcule ensuite, sur ce flux unique, la normale 1961–1990 de chaque station pour chaque mois, l'écart des mois demandés, la moyenne des stations par zone et par mois, puis le classement. Le calcul est donc fait côté client, après rapatriement des documents.

Ce tri explicite rend les résultats déterministes : il fixe l'ordre des valeurs additionnées, et les égalités du classement sont départagées par zone, année et mois. `weather.py` écrit un JSON aux clés triées. Le script `scripts.distributed.compare` compare automatiquement une F1 ciblée et une F3 multi-zone entre les profils local, `hashed` et `zoned` ; la F3 touche

plusieurs shards. La comparaison porte sur les réponses canoniques, après retrait du champ `profile` de l'enveloppe.

4.5 Perte d'une instance

Chaque requête passe par la fonction `run()` de `common.py` avant d'atteindre les données. Elle détermine d'abord le shard qui possède chaque station nécessaire, sans lire les observations : elle compare le hash de la station (profil `hashed`) ou le couple zone, station (profil `zoned`) aux plages de `config.chunks`. Elle contacte ensuite directement chaque shard avec un ping de 800 ms ; c'est la seule connexion qui contourne `mongos`. Si un shard requis est éteint, le service n'est pas exécuté et l'utilisateur reçoit une enveloppe qui refuse le résultat et décrit ce qui manque :

```
{"status": "UNAVAILABLE", "complete": false,
 "missing": {"shards": [...], "n_stations": ...,
             "examples": [...], "areas": [...]},
 "message": "Résultat refusé: des observations requises
             sont injoignables."}
```

Si un shard tombe pendant l'exécution, l'erreur de MongoDB est rattrapée et la réponse porte le statut `ERROR`, toujours avec `complete: false`. Un résultat partiel n'est jamais présenté comme complet.

Le script `failure_demo.py` réalise la démonstration demandée. Après `cluster.py kill shard2`, il choisit une station stockée sur `shard2RS`, lance F1 sur cette station puis F3 sur les deux zones, et affiche les deux enveloppes `UNAVAILABLE`. La commande `cluster.py start --shards 2` relance ensuite le shard. En profil `zoned`, une F3 limitée au Michigan reste servie et complète quand `shard2RS` est éteint.

Pendant la panne, notre prototype ne récupère rien : les observations du shard éteint sont inaccessibles jusqu'à son redémarrage. Au redémarrage, MongoDB rejoue son journal, et les écritures déjà acquittées ne sont pas perdues. Un déploiement de production ferait de chaque shard un replica set de trois membres. Les deux membres restants formeraient une majorité et éliraient un nouveau primaire en quelques secondes ; F1 et F3 resteraient complets, et une écriture acquittée par la majorité survivrait à la perte du primaire. Il faudrait aussi répliquer le config server et lancer plusieurs `mongos`. Trois shards représenteraient alors neuf nœuds de données. Notre déploiement sur un poste ne peut démontrer ni élection, ni partition réseau.

La protection a une limite. Les métadonnées lues avant l'exécution (`stations` sur le primary shard, `config.chunks` sur le config server) le sont hors du bloc qui rattrape les erreurs. Si `shard1RS` ou le config server est éteint, l'utilisateur reçoit une exception Python au lieu de l'enveloppe. L'échec reste explicite, mais il n'est pas décrit.

4.6 Limites du prototype distribué

- Aucune réplication : chaque shard est un replica set d'un membre, et la perte d'un shard rend ses observations indisponibles.
- Points uniques de défaillance : un seul `mongos`, un config server d'un membre, et un primary shard qui porte `stations`, `history` et `batches`.

- F3 rapatrie les documents bruts vers l'application. Une agrégation `$group` par station et par mois calculerait des résultats partiels sur chaque shard et réduirait les transferts.
- Tous les processus partagent un seul poste : les gains ou pertes mesurés ne se transposent pas directement à un cluster de machines.

5 Ce que disent les hivers

Notre hypothèse de départ était simple : il neige moins qu'avant. Les données la corrigent. La neige tombe toujours, parfois plus fort ; c'est sa tenue au sol qui recule, et elle recule le plus là où l'hiver est le plus doux.

Des hivers d'enfance vraiment blancs. La station de Grosse Pointe Farms (`USC00203477`) est à 3,7 km de Grosse Pointe Woods, où l'un de nous a grandi entre 2000 et 2011. Le souvenir d'hivers blancs n'est pas une illusion : ces onze hivers comptent autant de jours de neige au sol que la normale 1961–1990, après une décennie 1990 nettement plus pauvre. C'est après 2011 que tout bascule (tableau 5). À l'aéroport de Détroit, mieux renseigné, la chute est plus modérée, mais la neige tombée, elle, ne baisse presque pas. La figure 2 montre les 59 hivers de cette station.

Moyenne décembre–février	1961–1990	1990–2000	2000–2011	2011–2020
Jours de neige au sol, Grosse Pointe Farms	49	38	48	29
Jours de neige au sol, Detroit Metro	45	31	46	39
Neige tombée, Detroit Metro (mm)	775	745	1 005	965

Tableau 5: Hivers complets (règle 3/5) aux deux stations les plus proches de Grosse Pointe Woods. La station de Grosse Pointe Farms n'a que 5 hivers complets après 2011 : la tendance y est nette, mais repose sur peu d'hivers.

Autant de neige, mais elle tient moins. Une station ne fait pas une région. Sur les 38 stations du Michigan qui mesurent à la fois la neige tombée et la neige au sol sur les deux périodes, la neige tombée ne bouge pas (1 545 puis 1 554 mm par hiver). Le nombre de jours de neige au sol passe pourtant de 73 à 67, soit 4,75 jours par 10 cm de neige au lieu de 5,28 : une baisse de 10 %, observée à 27 stations sur 38. Les fortes chutes, elles, augmentent : sur 64 stations, F2 compte 11 % de jours de plus de 15 cm par an. L'hiver du Michigan n'est pas moins neigeux, il est moins durable.

Le sud de l'Allemagne perd plus vite. À l'échelle des zones, F3 compare chaque mois d'hiver de 1991–2020 à sa normale 1961–1990. Le Michigan perd 8 % de ses jours de neige au sol (environ 6 jours par hiver) pour un réchauffement de 1,4 °C des minimales. Le sud de l'Allemagne, qui ne se réchauffe que de 1,0 °C, en perd 27 % (environ 12 jours), et la perte s'accélère : −22 % sur 1991–2005, −32 % sur 2006–2020.

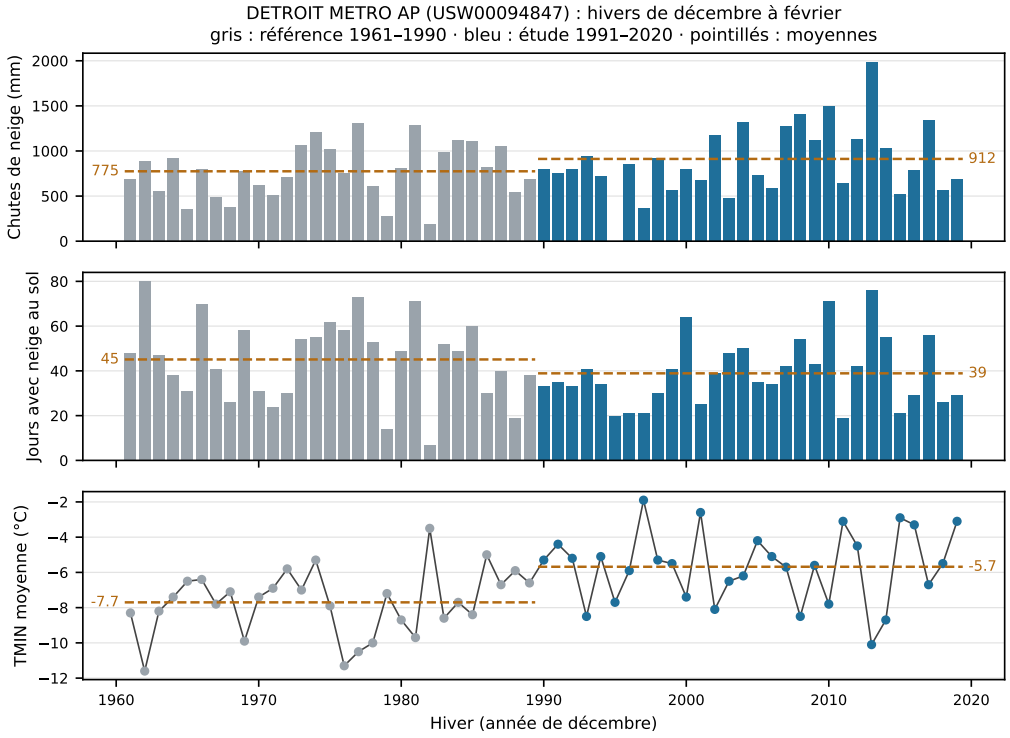


Figure 2: Hivers (décembre–février) à Detroit Metro, calculés par F1. Les hivers qui ne respectent pas la règle 3/5 sont omis.

Pourquoi : l'hiver allemand a franchi zéro. La réponse tient au point de départ. Sur 66 stations de chaque zone, la journée d'hiver moyenne du Michigan passe de $-5,7$ à $-4,6$ °C : plus chaude, mais encore loin du dégel, avec 63 % des jours sous -2 °C. Celle du sud de l'Allemagne passe de $-0,5$ à $+0,7$ °C : elle a franchi le point de fusion, et la part des jours au-dessus de $+2$ °C est montée de 31 à 40 %. Le même réchauffement fond plus de neige là où l'hiver était déjà au bord du zéro.

Ce qui pourrait nous tromper. L'objection la plus sérieuse est l'arrivée massive de stations bénévoles au Michigan après 2008 : comparer des moyennes sur toutes les stations reviendrait à comparer deux réseaux différents. Tous les chiffres de cette section portent donc sur les mêmes stations aux deux périodes. Restent deux limites que nous ne levons pas : la façon de mesurer la neige a pu changer d'un observateur à l'autre, et aucun test de significativité n'a encore été fait.

6 Conclusion

Dans ce Projet ...

Références

[1] M. J. Menne, I. Durre, R. S. Vose, B. E. Gleason et T. G. Houston. An overview of the Global Historical Climatology Network-Daily database. *Journal of Atmospheric and Oceanic Technology*, 29(7):897–910, 2012.

[2] World Meteorological Organization. *WMO Guidelines on the Calculation of Climate Normals*. WMO-No. 1203, Genève, 2017.

[3] R. Cifelli, N. Doesken, P. Kennedy, L. D. Carey, S. A. Rutledge, C. Gimmestad et T. Depue. The Community Collaborative Rain, Hail, and Snow Network: informal education for scientists and citizens. *Bulletin of the American Meteorological Society*, 86(8):1069–1077, 2005.

[4] A. M. G. Klein Tank et al. Daily dataset of 20th-century surface air temperature and precipitation series for the European Climate Assessment. *International Journal of Climatology*, 22(12):1441–1453, 2002.

[5] Natural Earth. 1:10m Cultural Vectors, Admin 1 – States, Provinces; 1:10m Physical Vectors, Lakes. Version 5.1.2. `https://www.naturalearthdata.com`, domaine public.

A Anomalies relevées dans les données

Les fréquences portent sur les 31,6 millions d'observations chargées (1961–2020, 1 352 stations). Les exemples sont des lignes réelles des fichiers `by_station` de la NOAA, au format `station,date,élément,valeur,M,Q,S,heure` ; elles ont été relues dans les fichiers d'origine. Comptages produits par `scripts/coverage_report.py` (`results/coverage/anomalies.json`).

A1. Valeurs signalées par un drapeau qualité. 56 360 valeurs (0,18 %) portent un drapeau qualité, dont 47 431 le drapeau `I` (échec d'un contrôle de cohérence interne, par exemple TMAX inférieur à TMIN). Le Michigan en concentre la grande majorité.
`GM000002698,19620128,TMIN,4,,I,E,` (Karlsruhe, 0,4 °C signalé `I`).
Traitement : valeur conservée et traçable, exclue des analyses.

A2. Valeurs physiquement impossibles. 63 valeurs portent le drapeau `X` (hors limites) et 255 le drapeau `G` (valeur aberrante). Le cas le plus net est Big Rapids (Michigan) le 11 janvier 1982 : 2 032 mm de pluie, 2 606 mm de neige et 38,9 °C le même jour d'hiver.

```
USC00200779,19820111,PRCP,20323,,X,0,2400
USC00200779,19820111,SNOW,2606,,X,0,
USC00200779,19820111,TMAX,389,,G,0,2400
```

Nous n'avons trouvé aucune valeur hors de nos bornes de plausibilité sans drapeau NOAA.
Traitement : comme A1.

A3. Précipitations « manquantes, présumées nulles » (drapeau P). 1 466 628 valeurs PRCP ou SNOW portent le drapeau de mesure `P`, sur 298 stations, toutes au Michigan (aucune en Allemagne).
`USC00200032,19820105,PRCP,0,P,,0,2400` (Adrian, Michigan).
Traitement : traitées comme manquantes, jamais comme un vrai zéro. Les traces (`T`, valeur 0) sont comptées comme jour de précipitation de quantité négligeable.

A4. Neige au sol en été, sans drapeau. 4 valeurs de hauteur de neige positives en juin, juillet ou août, toutes les 1er et 2 juin 1962, à quatre stations allemandes différentes (10 à 90 mm).
`GME00102412,19620602,SNWD,40,,E,` (Kempten, 40 mm).
Traitement : conservées. Plusieurs stations voisines les mêmes jours évoquent plutôt une vraie chute de neige tardive qu'une erreur : c'est le cas qu'un événement régional (F4) doit distinguer d'un extrême isolé.

A5. Un même site sous deux identifiants. 95 paires de stations sont à moins de 1 km l'une de l'autre. À Gelbelsee (Bavière), `GMM00010777` (1988–2003) et `GME00122734` (depuis 1998) sont à 250 m. Sur les 36 jours de 2003 où les deux donnent TMAX, 33 valeurs sont identiques et 3 diffèrent.
`GMM00010777,20030518,TMAX,158,,S,` `GME00122734,20030518,TMAX,158,,E,`
`GMM00010777,20030525,TMAX,181,,S,` `GME00122734,20030525,TMAX,215,,E,`
Traitement : stations gardées séparément ; dans F3 et F4, une paire ne compte que pour un seul site.

A6. Séries qui commencent, s'arrêtent ou s'interrompent. Au Michigan, 888 stations commencent après janvier 1991 et 609 s'arrêtent avant décembre 2020 (67 et 108 en Allemagne). Parmi les stations à longue série, la plus longue interruption est à Tirschenreuth-Lodermühl : 1 947 jours sans donnée.
`GME00129298,20021031,TMAX,60,,E,` puis `GME00129298,20080301,TMAX,85,,E,`
L'inventaire peut masquer ces trous : il annonce TMIN de 1943 à 2026 pour Michigamme (`USC00205403`), alors que le fichier s'arrête en 1949 et ne reprend qu'en 2022.
Traitement : statut « longue série » mesuré mois par mois (règle 3/5 de l'OMM), pas d'après l'inventaire.

B Référence des services

Tous les services s'appellent par `python3 -m weatherscope <commande>` ; chacun accepte `-json` et `-uri` pour viser le serveur unique ou le cluster. Les exemples ci-dessous sont des sorties réelles sur nos données.

B.1 F1 : explorer une station

Entrées. `stations` : une zone (nom, rectangle `-bbox` ou rayon `-near`), éventuellement un élément ou un nom. `f1` : une station, un intervalle de dates, un découpage par mois, année

ou hiver, et les éléments voulus.

Sorties. La liste des stations de la zone ; puis, pour chaque élément et chaque période : la disponibilité (jours attendus, jours utilisables, couverture, validité selon la règle 3/5), la qualité (valeurs signalées, drapeaux `P` et `T`) et des statistiques calculées sur les seules valeurs utilisables.

Fonctionnement. Une seule agrégation MongoDB, qui passe par l'index (station, élément, date) et regroupe par élément et par période. Les 30 hivers d'une station s'obtiennent en un peu plus d'une seconde.

Exemple. L'hiver 2013–14 à Detroit Metro (tableau 6) est complet et sans drapeau : 996 mm de neige en janvier et de la neige au sol tous les jours de janvier et de février. À l'inverse, la station bénévole de Woodhaven (`US1MIWY0011`) affiche 683 mm de pluie en 2010, mais avec seulement 17,5 % de jours renseignés : F1 marque l'année comme non valide, et ce total ne doit pas être comparé.

Mois	Couverture	TMIN moy. (°C)	TMIN min (°C)	Neige (mm)	Jours de neige au sol
2013-12	100 %	$-6,2$	$-14,9$	396	17
2014-01	100 %	$-13,0$	$-25,5$	996	31
2014-02	100 %	$-11,1$	$-20,5$	593	28

Tableau 6: Extrait de F1 pour `USW00094847` (Detroit Metro), du 1/12/2013 au 28/2/2014.

B.2 Traçabilité

Entrées. Des identifiants `station:date:élément`, ou une station, une période et des éléments.

Sorties. Chaque relevé tel qu'il est stocké : valeur brute, valeur convertie, drapeaux, heure d'observation, fichier source et lot d'import ; avec `-history`, les versions remplacées par un import ultérieur.

Exemple. Le minimum de $-25,5$ °C du tableau 6 renvoie à un seul document : valeur brute -255, aucun drapeau, source `W`, relevé à 24 h, fichier `USW00094847.csv.gz`, lot `2026-11-08-initial`. C'est la ligne `USW00094847,20140106,TMIN,-255,,,W,2400` du fichier NOAA.

B.3 F2 : épisodes prolongés

Entrées. Une zone ou une liste de stations, une période, un élément, une durée minimale et une règle : seuil fixe (`-above`, `-below`) ou percentile du climat propre à la station sur 1961–1990 pour le même mois (`-pct-above`, `-pct-below` ; pour la pluie et la neige, calculé sur les jours d'au moins 1 mm). Les stations sans assez de valeurs de référence sont écartées et comptées.

Sorties. Pour chaque épisode : station, début, fin, durée, intensité (écart moyen au seuil), valeur extrême, cumul pour la pluie et la neige, ce qui l'a ouvert et fermé (règle non remplie, jour manquant, jour signalé, limite de la requête) et les paramètres à passer à `trace`.

Exemple. Avec TMAX ≥ 30 °C pendant au moins 5 jours, l'été 2003 donne 181 épisodes sur 160 stations du sud de l'Allemagne ; les plus longs couvrent du 1er au 14 août, avec 40,2 °C à Karlsruhe et à Fribourg (1,5 s). À Détroit, avec TMAX ≤ 0 °C, la vague de froid du 30 décembre 2013 au 9 janvier 2014 sort comme un seul épisode de 11 jours. Au Michigan, sur 1991–2020, 85 épisodes de chaleur se terminent sur un jour manquant et 38 sur un jour signalé ; à `USC00200710`, le 15 août 1995 affiche 32,2 °C avec le drapeau `I` et ne prolonge donc pas l'épisode.

B.4 F3 : mois anormaux

Entrées. Un élément, une ou deux zones, les années étudiées (défaut 1991–2020), la référence (défaut 1961–1990), éventuellement des mois du calendrier (par exemple 12, 1, 2), la taille des cases (défaut 1°), le nombre minimal d'années valides pour une normale (défaut 24) et la part minimale de cases renseignées (défaut 50 %).

Sorties. Pour chaque zone, les mois classés par anomalie standardisée : anomalie, normale, pourcentage de la normale pour la pluie et la neige, nombre de stations et de cases ; pour chaque mois, les stations avec leur valeur, leur normale et leur anomalie, et les dates à tracer. Les mois écartés faute de couverture sont listés.

Exemple. Sur TMAX, les mois en tête sont mars 2012 au Michigan ($+8,1$ °C, anomalie standardisée 3,5, 76 stations dans 27 cases sur 31) et août puis juin 2003 en Allemagne ($+7,0$ et $+6,9$ °C). Une requête sur les deux zones et 60 ans prend environ 2 minutes.